REPORT FOR MAD 2 PROJECT

Author

Name - Saumya Sarkar

Roll No. - 21F3001551

Student email - 21f3001551@ds.study.iitm.ac.in

About me

I am passionate about data science and data analytics.I am currently pursuing the BS degree in Data Science and Application from IIT Madras.

Description

`QuizDeck` is an application where users can sign up for taking online quizzes, and prepare for exams. It has quizzes on various topics, users can browse and take quizzes and see their results after the quiz. Users can also track their performance over time by means of the summary charts provided.

AI/LLM was only used for CSS styling to improve aesthetics. There was no usage of AI in generating backend coding logic.

Approach

1. Created the DB models, separate models to store users, quizzes etc.
2. Created API and implemented **RBAC** to secure the API via flask session based token.
3. Started working on frontend to create interactive and user friendly **vue 3 pages**, CRUD operation for Admin, implemented robust validation.
4. Implemented the quiz taking for users, where intermediate answers and duration will be persistent, so users can resume quizzes in case of any connection issue.
5. Created vue 3 pages where Users can see results and other performance related charts.
6. Implemented caching to improve api performance, along with backend asynchronous tasks like monthly performance report mail to users, CSV report export for admin and daily reminders about new quizzes for users in a dedicated google space called `QuizDeck`.

Technologies used

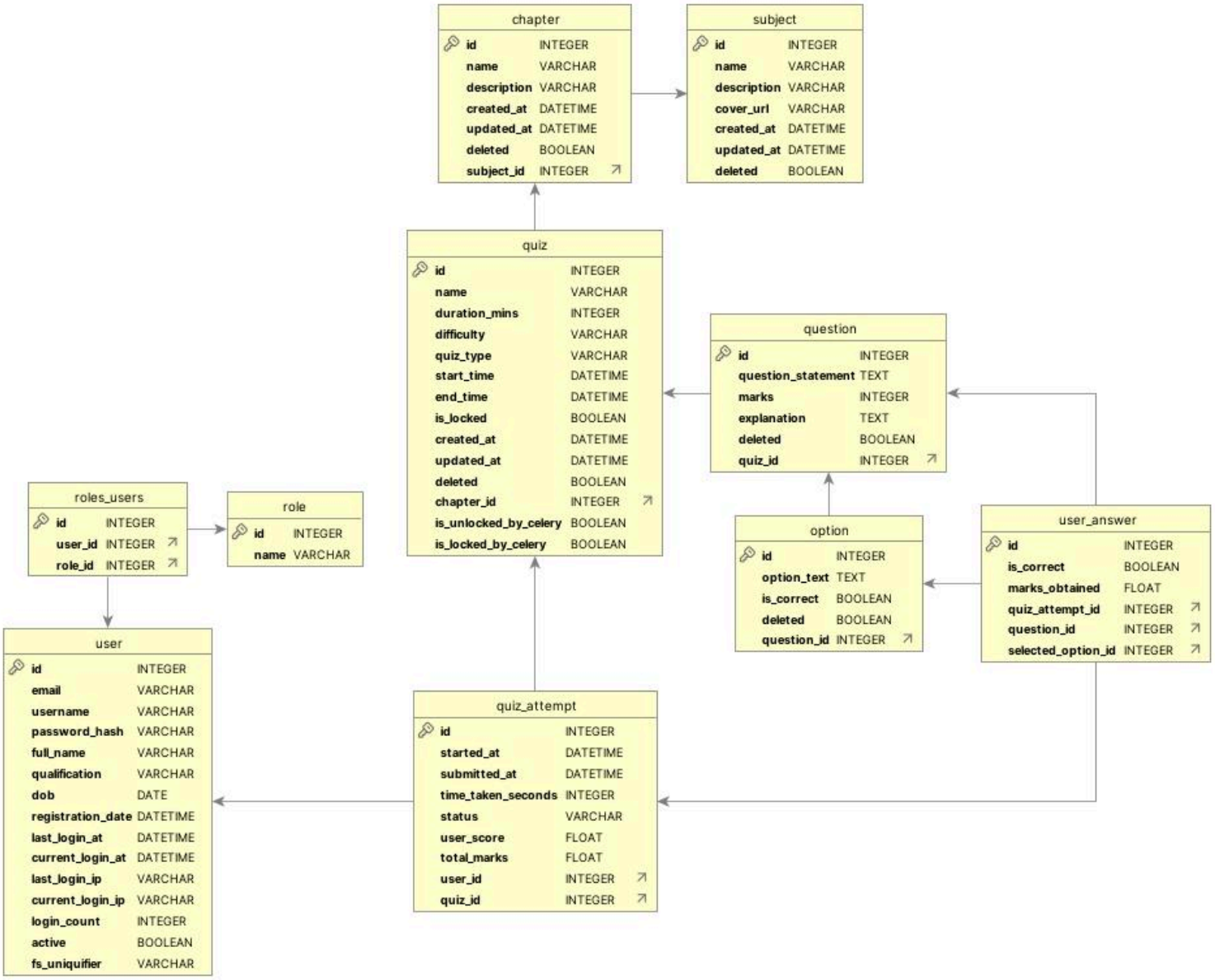
- **Flask -** A lightweight backend framework for building web applications with Python.

- **Flask-Sqlalchemy -** ORM tool for database interactions.
- **Sqlite -** Database management system for storing application data.
- **Vue 3** - A progressive JavaScript framework for building user interfaces and enhancing interactivity, complemented by Bootstrap and custom CSS for styling.
- **Flask Security** - An extension that provides tools for managing user sessions and authentication using flask session based tokens, comes with
- **Flask Restful -** A Python library that helps with the creation of RESTful APIs along with reqparse for backend validation and parsing incoming requests.
- **Celery** - An asynchronous message queue that enables the execution of background tasks and scheduled tasks.
- **Redis** - An in-memory data structure store used as a caching database to optimize api performance and store messages and results for asynchronous tasks for celery.
- **Argon2** - A secure password hashing algorithm recommended for use in Flask applications to protect user passwords
- **VueChartJS** - Used for creating interactive charts and visualizations for admin and user d
- **Mailhog** - Testing email communication.
- **VueToastification** - For showing relevant message toasts to the user
- **Flask Caching** - For caching api end points.

DB Schema Design

- Created `user` table to store users, `role` table to store roles i.e Admin or User and an association table `roles_users.`
- There is a hierarchy of tables from `subject` > `chapter` > `quiz` > `question`
- These tables have a `deleted` column which enables admin to **soft delete** items with the possibility of restore when needed and also these have `created_at` and `updated_at` columns to track changes to the data
- There is a separate `option` table to store options to keep the database normalized.
- `quiz_attempt` table stores the quiz attempts by the user along with status, which becomes **in progress** when the user starts a quiz, these **enable resuming quiz**.
- Finally the answers are saved during the quiz and after submission in the `user_answer` table.
- There are two columns named `is_locked_by_celery` and `is_unlocked_by_celery`, which enables admin to track the **automatic quiz locking and unlocking by celery**.

DB Schema Design - ER Diagram



API Design

- API for user registration and login and real time validation of user inputs.
    - /api/register - user registration
    - /api/login - user login
    - /api/qualifications - fetching valid qualifications from backend
    - /api/check-username - to check if username already exists
    - /api/check-email - to check if email already exists
    - /api/user-details - to fetch user details to **rehydrate user vuex store on refresh**

- API for admin to handle CRUD operations on Subject, Chapter, Quiz and Question
    - /api/subject , api/chapter, api/quiz, api/question - To **Read** data from backend
    - /api/subject/update, api/chapter/update, api/quiz/update, api/question/update - To **Create and Update** respective entities

    - /api/subject/delete, api/subject/delete, api/subject/delete, api/subject/delete - To **Delete** respective entities

- API for quiz taking by user
    - /api/quiz/start - Starting a quiz, check if quiz is locked or not
    - /api/quiz/data - Fetching quiz questions upon successful start
    - /api/quiz/save-answer - Saving intermediate progress
    - /api/quiz/submit - Submission of quiz
    - /api/quiz/result - Calculate and fetch results after completing quiz

- API for reading past user attempts
    - /api/user/quiz-attempts - fetching quiz history for users

- API for retrieving data required for **analytics and creating summary charts**
    - /api/admin/analytics - Analytics for admin
    - /api/user/analytics - Analytics for user

- API for backend task - **Export CSV Report for Admin**
    - /api/csv-export/generate - Initiating CSV report creation
    - /api/csv-export/status - Checking if the report is ready - **Polling**
    - /api/csv-export/download/<string:task_id> - Sending download once its ready

Architecture

- Backend Structure - The app.py is in the backend root folder, `routes` folder has all the **controllers,** the backend root folder contains other various `configurations`. The `static` folder has CSV reports and uploaded images.
- Frontend Structure - The `views` , `components`, `router` and `store` are located in the `src` folder and the `public` folder contains the `index.html`.

Additional Features

- ❖ Ability to **resume quizzes** in case of network disruption.
- ❖ **Robust validation** in the frontend and backend
- ❖ **Automatic quiz locking and unlocking** by celery upon expiration or start
- ❖ Detailed **analytics and charts** for admin and users
- ❖ **Auto refresh** page when a quiz unlocks to enable start button
- ❖ **Glassmorphic** design
- ❖ Well-designed **HTML reports** for Monthly activity reports

Video

Video Link - QuizDeck Demo Video.mp4 (*Please download the video for a better viewing experience.*)